

ПЕРВОЕ ВЫСШЕЕ ТЕХНИЧЕСКОЕ УЧЕБНОЕ ЗАВЕДЕНИЕ РОССИИ



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное бюджетное образовательное учреждение высшего образования

САНКТ-ПЕТЕРБУРГСКИЙ ГОРНЫЙ УНИВЕРСИТЕТ ИМПЕРАТРИЦЫ ЕКАТЕРИНЫ II

Кафедра информатики и компьютерных технологий

ПРАКТИЧЕСКАЯ РАБОТА №1

По дисциплине: Практические навыки и опыт анализа данных с помощью Python

(наименование учебной дисциплины согласно учебному плану)

Тема работы: Введение в Python

Выполнил: студент гр.

ИТ-25-3 (ИАС-25-1)
(шифр группы)

(подпись)

Черников В.М.
(Ф.И.О.)

Оценка: _____

Дата: _____

Проверил

руководитель работы:

доцент
(должность)

(подпись)

Кочнева А.А.
(Ф.И.О.)

Санкт-Петербург 2026

Введение

Язык программирования Python на сегодняшний день является одним из наиболее востребованных и универсальных инструментов в IT-сфере. Понимание его базового синтаксиса, встроенных структур данных и принципов работы с внешними источниками информации (файлами) является фундаментом для дальнейшей разработки любого программного обеспечения — от простых скриптов автоматизации до сложных веб-приложений и систем анализа данных.

Калькулятор оценок студентов

Проанализируем код, представленный в задании.

- Функция `grade_calculator()` — интерактивный калькулятор средней оценки с циклом повтора (Скриншот 1)
- Функция `calculate_average_grade()` — вычисление среднего балла из словаря оценок. (Скриншот 2)
- Функция `print_student_info()` — вывод средних оценок всех студентов на экран. (Скриншот 2)
- Функция `read_students_from_file_safe()` — безопасное чтение файла с отловом ошибок формата и отсутствия файла. (Скриншот 2)
- Функция `main()` — точка входа, объединяющая чтение файла и вывод данных. (Скриншот 3)

```

1 def grade_calculator():
2     while True:
3         students = {}
4         student_name = input("Введите имя студента: ")
5
6         try:
7             num_subjects = int(input("Введите количество предметов: "))
8         except ValueError:
9             print("Ошибка: введите число.")
10            continue
11
12        for i in range(num_subjects):
13            subject = input(f"Введите название предмета {i + 1}: ")
14            try:
15                grade = float(input(f"Введите оценку за {subject}: "))
16                students[subject] = grade
17            except ValueError:
18                print("Ошибка: введите числовое значение оценки.")
19                students[subject] = 0.0
20
21
22        total_grades = sum(students.values())
23        average_grade = total_grades / num_subjects if num_subjects > 0 else 0
24
25        if average_grade ≥ 90:
26            grade_label = "Отлично"
27        elif average_grade ≥ 75:
28            grade_label = "Хорошо"
29        elif average_grade ≥ 60:
30            grade_label = "Удовлетворительно"
31        else:
32            grade_label = "Неудовлетворительно"
33
34        print(f"Студент: {student_name}, Средняя оценка: {average_grade:.2f} - {grade_label}")
35
36
37        repeat = input("Хотите ввести данные для другого студента? (да/нет): ")
38        if repeat.lower() ≠ "да":
39            print("Завершение работы программы.")
40            break

```

Скриншот 1

```

42 def read_students_from_file_safe(filename):
43     students = {}
44     try:
45         with open(filename, 'r', encoding='utf-8') as file:
46             for line in file:
47                 try:
48                     name, subject, grade = line.strip().split(';')
49                     grade = int(grade)
50                     if name not in students:
51                         students[name] = {}
52                     students[name][subject] = grade
53                 except ValueError as ve:
54                     print(f"Ошибка в строке '{line.strip()}': {ve}. Проверьте правильность формата.")
55                 except Exception as e:
56                     print(f"Неизвестная ошибка: {e}. Проверьте строку '{line.strip()}'.")
57     except FileNotFoundError:
58         print(f"Файл '{filename}' не найден.")
59     except IOError as io_error:
60         print(f"Ошибка ввода-вывода: {io_error}. Проверьте ваш файл.")
61     return students
62
63
64 def calculate_average_grade(student_grades):
65     if not student_grades:
66         return 0
67     total = sum(student_grades.values())
68     return total / len(student_grades)
69
70
71 def print_student_info(students):
72     for student, grades in students.items():
73         average = calculate_average_grade(grades)
74         print(f"Студент: {student}, Средняя оценка: {average:.2f}")
75

```

Скриншот 2

```

77 def main():
78     filename = 'students.txt'
79     students = read_students_from_file_safe(filename)
80     print_student_info(students)
81
82
83 if __name__ == "__main__":
84     main()

```

Скриншот 3

Калькулятор голосований

Создать программу, управляющую голосованием за кандидатов на выборах, которая хранит данные о кандидатах в словаре и сохраняет их в текстовом файле.

- `add_candidate()` — добавляет кандидата . (Скриншот 1)
- `vote()` — начисляет голос с проверкой существования кандидата .(Скриншот 4)
- `save_results()` — безопасно сохраняет итоги в файл (защищает от ошибок доступа). (Скриншот 4)
- `display_results()` — выводит отсортированный отчет с процентами и защитой от деления на ноль. (Скриншот 5)

```

1 candidates = {}
2
3 def add_candidate(name):
4     """Добавляет кандидата в словарь со значением 0 (голоса)."""
5     if name not in candidates:
6         candidates[name] = 0
7     else:
8         print(f"Кандидат {name} уже существует.")
9
10 def vote(name):
11     """Увеличивает количество голосов. Если кандидата нет — выводит сообщение."""
12     if name in candidates:
13         candidates[name] += 1
14     else:
15         print(f"Ошибка: Кандидат '{name}' не найден.")
16
17 def save_results(filename):
18     """Сохраняет текущие данные о кандидатах в файл."""
19     try:
20         with open(filename, 'w', encoding='utf-8') as f:
21             for name, votes in candidates.items():
22                 f.write(f"{name};{votes}\n")
23             print("Результаты сохранены.")
24     except IOError as e:
25         print(f"Ошибка записи: {e}")

```

Скриншот 4

```

27 def display_results():
28     """Выводит процент голосов каждого кандидата."""
29     total_votes = sum(candidates.values())
30     if total_votes == 0:
31         print("Голосов еще нет.")
32     return
33
34     print("--- Результаты голосования ---")
35     for name, votes in candidates.items():
36         percentage = (votes / total_votes) * 100
37         print(f"{name}: {votes} голосов ({percentage:.2f}%)")
38

```

Скриншот 5

Заключение

Выполнение самостоятельных заданий (Калькулятор голосования, Система учета книг, Учет расходов) продемонстрировало, что, опираясь на базовый синтаксис Python, можно создавать полноценные прикладные утилиты для структурирования, анализа и сохранения информации. Понимание принципов работы с файловой системой и грамотная обработка исключений являются критически важным этапом, который позволяет перейти от написания «учебных» алгоритмов к разработке реального, стабильного программного обеспечения, готового к использованию конечным пользователем.